การสร้างประเภทข้อมูลใหม่ด้วย **คลาส**

ภาควิชาวิศวกรรมคอมพิวเตอร์
จุฬาลงกรณ์มหาวิทยาลัย
๒๕๕๙

หัวข้อ

- การสร้างประเภทข้อมูลใหม่ด้วยคลาส
- องค์ประกอบของคลาส
- การสร้างและการใช้อ็อบเจกต์
- ตัวอย่าง

ประเภทข้อมูล

- ที่ผ่านมา ถ้าข้อมูลเป็น
  - จำนวน ใช้ `int` หรือ `float`
  - ข้อความ ใช้ `str`
  - จริง/เท็จ ใช้ `boolean`
  - กลุ่มข้อมูล ใช้ `list, set, tuple, dict`
  - ข้อมูลที่ประกอบด้วยข้อมูลย่อย ๆ ที่สัมพันธ์กัน
    - ใช้ list, tuple, dict

บัตรประชาชน
- หมายเลข
- ชื่อ-สกุล
- ที่อยู่
- วันเกิด
- ...

หนังสือ
- ชื่อ
- หมายเลข ISBN
- ราคา
- สำนักพิมพ์
- ...

`list` กับ `tuple` ต้องจำลำดับของข้อมูลย่อย (ไม่ค่อยสะดวก)

`dict` จำชื่อข้อมูล (สะดวกกว่า)

รายการของหนังสือ (List of Tuples)

หนังสือ
- ชื่อหนังสือ
- หมายเลข ISBN
- ราคา

แบบที่ 1.1 – ใช้ tuple แทนหนังสือ

```
books=[
        ("Data Science",   "149190142X", 28.79), \
        ("Learning Python","1449355730", 37.06), \
        ("Data Analysis",  "1449319793", 27.68)  ]
for book in books:
      if book[0] == "Data Analysis":
            print("Price = $", book[2])
```

ต้องจำเลขลำดับข้อมูลย่อย

ต้องจำเลขลำดับข้อมูลย่อย

รายการของหนังสือ (List of Tuples)

หนังสือ
- ชื่อหนังสือ
- หมายเลข ISBN
- ราคา

แบบที่ 1.2 – ใช้ tuple แทนหนังสือ

```
books=[
        ("Data Science",   "149190142X", 28.79), \
        ("Learning Python","1449355730", 37.06), \
        ("Data Analysis",  "1449319793", 27.68)  ]
for (title,isbn,price) in books:
    if title == "Data Analysis":
        print("Price = $", price)
```

ต้องจำลำดับข้อมูลย่อย

รายการของหนังสือ (List of Dicts)

หนังสือ
- ชื่อหนังสือ
- หมายเลข ISBN
- ราคา

แบบที่ 2 – ใช้ dict แทนหนังสือ

```
books=[
{"title":"Data Science", "isbn":"140142X","price": 28.79},
{"title":"Easy Python", "isbn":"14455730","price": 37.06},
{"title":"Big Data", "isbn":"14493793","price": 27.68}  ]
for book in books:
    if book["title"] == "Data Analysis":
        print("Price = $", book["price"])
```

ยังมีอีกวิธีหนึ่ง (คลาส)

ไม่ต้องจำลำดับข้อมูลย่อย แต่ต้องจำ key แทน

คลาสเพื่อสร้างประเภทข้อมูลที่ต้องการ

- ใช้คลาสประกาศองค์ประกอบของข้อมูลประเภทใหม่ว่าประกอบด้วยข้อมูลย่อย ๆ อะไรบ้าง

class

```
class Book:
  def __init__(self,title,isbn,price):
    self.title = title
    self.isbn  = isbn
    self.price = price
```

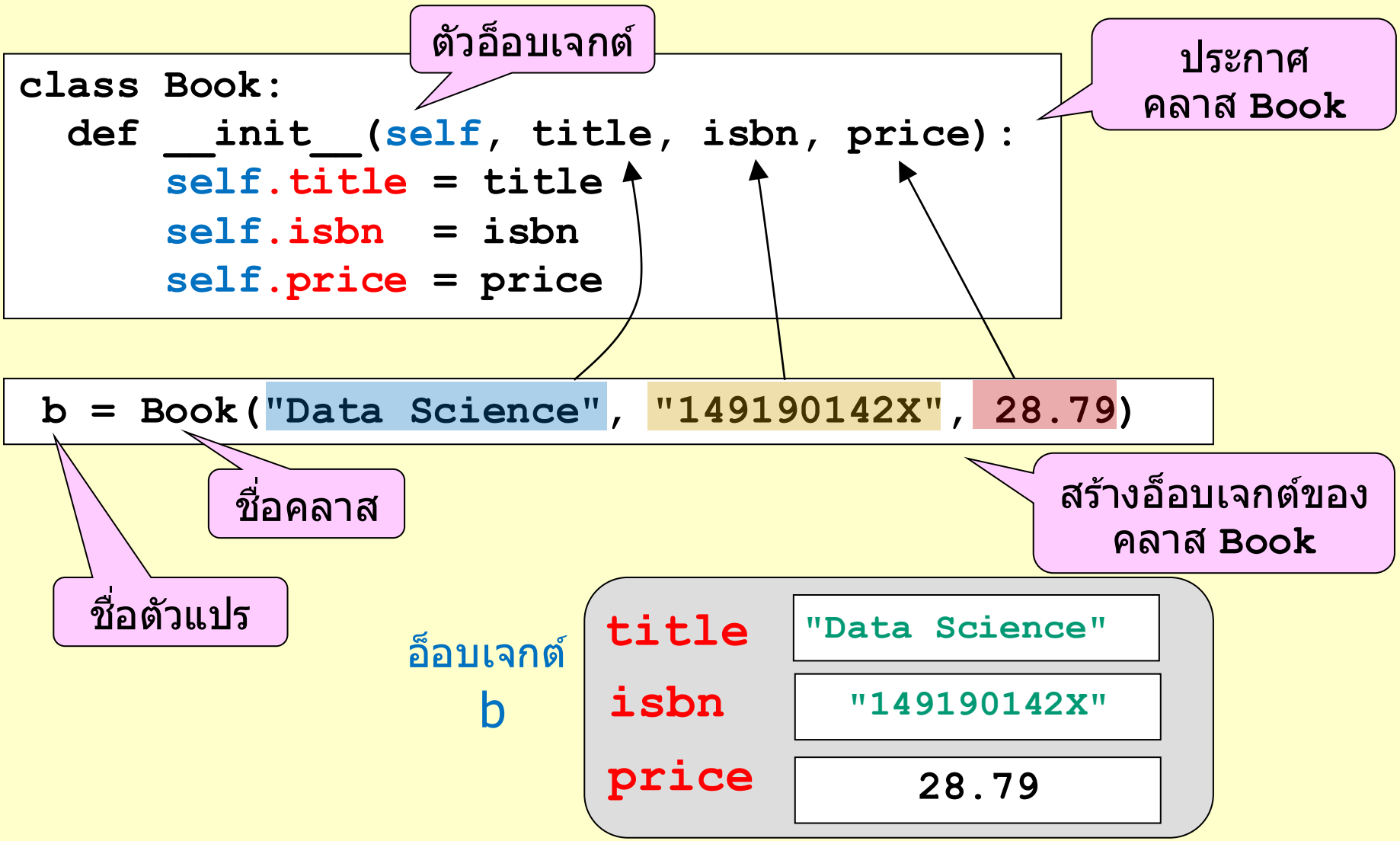
ตัวสร้าง (Constructor)

data members
attributes
fields

- เมื่อประกาศคลาสแล้ว สามารถสร้างที่เก็บข้อมูลเรียกว่าอ็อบเจกต์ (object) ด้วยการเรียกใช้ constructor (ดูหน้าถัดไป)

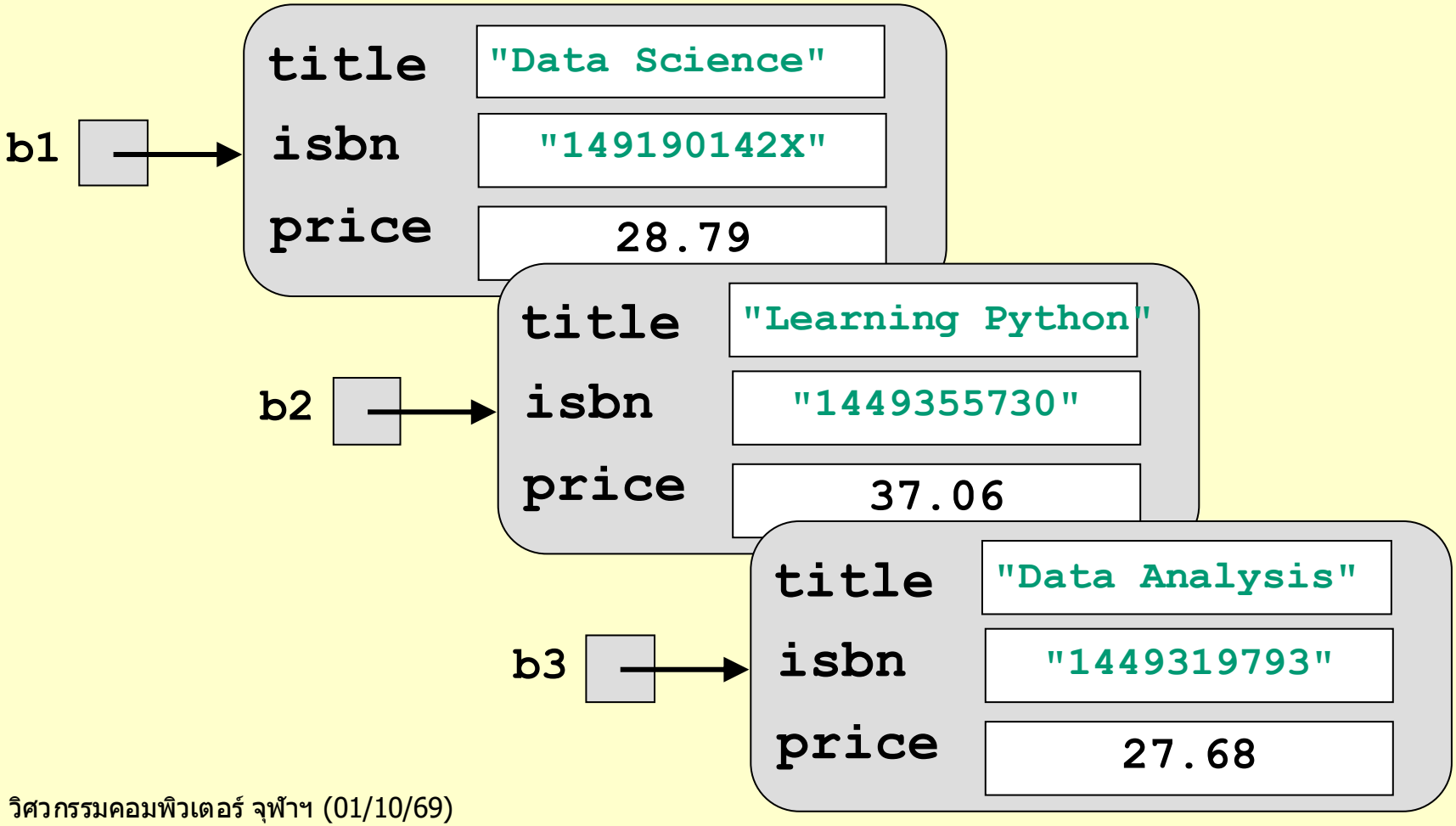
การสร้างอ็อบเจกต์

คลาส (class) คือประเภทข้อมูล อ็อบเจกต์ (object) คือตัวข้อมูล



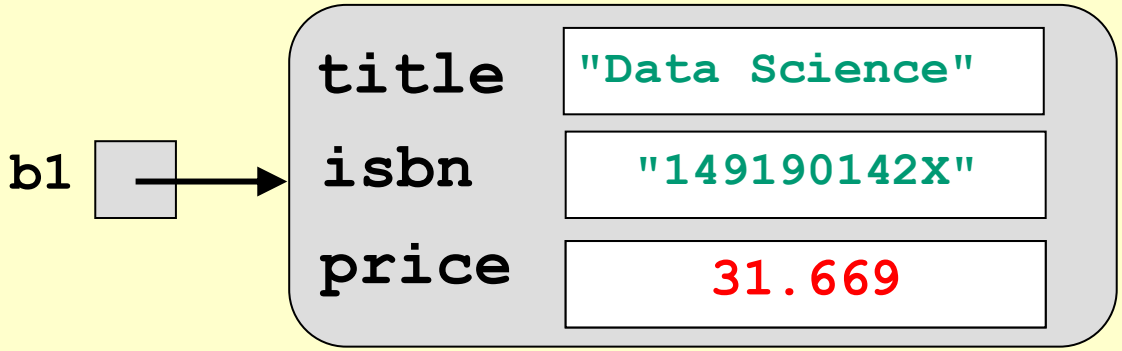
แต่ละอ็อบเจกต์มีตัวแปรประจำอ็อบเจกต์ของตัวเอง

```
b1 = Book("Data Science",   "149190142X", 28.79)
b2 = Book("Learning Python","1449355730", 37.06)
b3 = Book("Data Analysis",  "1449319793", 27.68)
```



การใช้/เปลี่ยนค่าของตัวแปรประจำอ็อบเจกต์

```
b1 = Book("Data Science", "149190142X", 28.79)
```




```
b1.price = b1.price * 1.10
```

การเรียกใช้งาน:
ชื่ออ็อบเจกต์ . ชื่อตัวแปร

b1.price อ่านว่า **ตัวแปร price ของอ็อบเจกต์ b1**

ตัวอย่าง : คลาสที่แทนประเภทข้อมูล

```
class Song:
    def __init__(self, title, artist, lyrics):
        self.title  = title
        self.artist = artist
        self.lyrics = lyrics
```

```
class Point:
    def __init__(self, x ,y):
        self.x = x
        self.y = y
```

```
class Date:
```

```
class ComplexNumber:
```

ลองเขียนเอง

ตัวอย่าง : Song และการพิมพ์เนื้อเพลง

```
class Song:
    def __init__(self, title, artist, lyrics):
        self.title  = title
        self.artist = artist
        self.lyrics = lyrics

s1 = Song("ช้าง","-",["ช้างช้างช้างช้างช้าง","น้องเคยเห็นช้างหรือเปล่า", \
     "ช้างมันตัวโตไม่เบา","จมูกยาวยาวเรียกว่างวง","มีเขี้ยวใต้งวงเรียกว่างา",\
     "มีหู>มีตา>หางยาว"])
s2 = Song("แมงมุมลาย","พลับ",["แมงมุมลายตัวโน้น", \
     "ฉันตะโกนเรียกมันทั้งวัน", "มันก็อยู่ตรงนั้น","ทุกวี่วันไม่มีพิษภัย"])

print("เพลง: ", s1.title, "\nศิลปิน: ", s1.artist)
for line in s1.lyrics:
    print(line)
print("เพลง: ", s2.title, "\nศิลปิน: ", s2.artist)
for line in s2.lyrics:
    print(line)
```

code ซ้ำ ควรแยกเป็นฟังก์ชัน

ตัวอย่าง : Song และฟังก์ชันการพิมพ์เนื้อเพลง

```
class Song:
    def __init__(self, title, artist, lyrics):
        self.title = title
        self.artist = artist
        self.lyrics = lyrics

def print_a_song(s):
  print("เพลง: ", s.title, "\nศิลปิน: ", s.artist)
  for line in s.lyrics:
      print(line)

s1 = Song("ช้าง","-",["ช้างช้างช้างช้างช้าง","น้องเคยเห็นช้างหรือเปล่า", \
     "ช้างมันตัวโตไม่เบา","จมูกยาวยาวเรียกว่างวง","มีเขี้ยวใต้งวงเรียกว่างา",\
     "มีหู>มีตา>หางยาว"])
s2 = Song("แมงมุมลาย","พลับ",["แมงมุมลายตัวโน้น", \
     "ฉันตะโกนเรียกมันทั้งวัน", "มันก็อยู่ตรงนั้น","ทุกวี่วันไม่มีพิษภัย"])

print_a_song(s1)
print_a_song(s2)
```

print_a_song เป็นบริการของ Song ควรอยู่กับ Song

ตัวอย่าง : Song และเมท็อดการพิมพ์เนื้อเพลง

```
class Song:
    def __init__(self, title, artist, lyrics):
        self.title  = title
        self.artist = artist
        self.lyrics = lyrics

    def print(self):
        print("เพลง: ", self.title, "\nศิลปิน: ", self.artist)
        for line in self.lyrics:
            print(line)

s1 = Song("ช้าง","-",["ช้างช้างช้างช้างช้าง","น้องเคยเห็นช้างหรือเปล่า", \
     "ช้างมันตัวโตไม่เบา","จมูกยาวยาวเรียกว่างวง","มีเขี้ยวใต้งวงเรียกว่างา",\
     "มีหู>มีตา>หางยาว"])
s2 = Song("แมงมุมลาย","พลับ",["แมงมุมลายตัวโน้น", \
     "ฉันตะโกนเรียกมันทั้งวัน", "มันก็อยู่ตรงนั้น","ทุกวี่วันไม่มีพิษภัย"])

s1.print() #เหมือน Song.print(s1)
s2.print() #เหมือน Song.print(s2)
```

เรียกฟังก์ชันของคลาสว่า method

การเรียกใช้งาน:
ชื่ออ็อบเจกต์ • ชื่อเมท็อด( )

ฟังก์ชัน vs เมท็อด

```
class Song:
    ...

    def print(self):
        print("เพลง: ", self.title, "\nศิลปิน: ", self.artist)
        for line in self.lyrics:
            print(line)
```

```
Song.print(s1)
```

เรียกใช้ฟังก์ชัน print ของคลาส Song ส่ง s1 ไปให้ self

```
s1.print()
```

เรียกใช้เมท็อด print ของอ็อบเจกต์ s1
ระบบจะรู้ว่า s1 เป็นอ็อบเจกต์ของคลาส Song ก็จะแปลงไปเป็น

เคยใช้ฟังก์ชันกับเมท็อดมามากมาย

functions

```
a = [1,2,3,4]
k = len(a)
print(a)
s = sum(a)
b = sorted(a)

# ฟังก์ชันใน module>อื่น
k = math.sin(1)
d = np.ndarray((2,3))
```

methods

```
a = [1,2,3,4]
a.sort()
a.reverse()
t = "aBc"
u = t.upper()
v = t.lower()
k = t.find("B")
s = set(a) # constructor
s = s.union([3,5])
```

คลาสนิยามข้อมูลย่อยและบริการ

```
class Song:
    def __init__(self, title, artist, lyrics):
        self.title  = title
        self.artist = artist
        self.lyrics = lyrics

    def print(self):
        print("เพลง: ", self.title, "\nศิลปิน: ", self.artist)
        for line in self.lyrics:
            print(line)

    def covered_by(self, new_artist):
        return Song(self.title, new_artist+" (cover)",
                    self.lyrics)

s1 = Song("HBD","?",["HBD2U","HBD2U", "HBD HBD", "HBD2..U"])
s2 = s1.covered_by("Bua")
s2.print()
```

คลาสประกอบด้วย ตัวสร้างเพื่อตั้งค่าของข้อมูลย่อย
และเมท็อดต่าง ๆ ที่ให้บริการการใช้งานข้อมูลย่อย

ตัวอย่าง : BankAccount

- คลาส BankAccount : ประกาศประเภทข้อมูล
  - แทนบัญชีธนาคาร
  - เก็บแค่หมายเลขบัญชี (acc_no) ชื่อบัญชี (acc_name) และยอดเงินปัจจุบัน (balance)

- เมท็อดสำหรับ BankAccount
  - ฝาก : deposit(...)
  - ถอน : withdraw(...)

ตัวอย่าง : BankAccount มีบริการฝาก/ถอน

```
class BankAccount:
    def __init__ (self, acc_no, acc_name, balance):
        self.acc_no   = acc_no
        self.acc_name = acc_name
        self.balance  = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

    def withdraw(self, amount):
        self.balance -= amount
        return self.balance

a1 = BankAccount("1-034-567-892", "ปราณี>รักเรียน", 500)
a1.deposit(1000)
a1.withdraw(150)
print("Account#", a1.acc_no, ">> balance =", a1.balance)
```

```
Account# 1-034-567-892 >> balance = 1350
```

ตัวอย่าง : BankAccount ตรวจยอดเงินก่อนฝากถอน

```
class BankAccount:
    def __init__ (self, acc_no, acc_name, balance):
        self.acc_no   = acc_no
        self.acc_name = acc_name
        self.balance  = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
        return self.balance

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
        return self.balance
```

ตัวอย่าง : เรียงลำดับบัญชีธนาคารตามยอดเงิน

- อ่านข้อมูลเข้ามาเก็บในลิสต์ของอ็อบเจกต์บัญชี
- เรียงลำดับลิสต์ของอ็อบเจกต์บัญชีตามยอดเงิน
- แสดงผลลัพธ์ของการเรียงลำดับ

```
class BankAccount :
   ...

def read_data(filename):
   ...

#------- Program start here --------
accounts = read_data("data_bank.txt") #accounts เป็น list
accounts.sort()                       # accounts = sorted(accounts)
print(accounts)                       # built-in function print
```

```
010-123 John Smith      1000.00
010-213 Andrew Gibson   1500.00
010-312 Suthep Sawasdi   800.00
...
```

จะเรียงลำดับด้วย **sort** หรือ **sorted** ได้ ต้องรู้วิธีเปรียบเทียบอ็อบเจกต์ →

จะแสดงด้วย **print** ได้ ต้องรู้วิธีเปลี่ยนอ็อบเจกต์เป็นสตริง →

เมท็อด __lt__ เพื่อเปรียบเทียบอ็อบเจกต์

```
class BankAccount :
    def __init__ (self, acc_no, acc_name, balance):
        self.acc_no   = acc_no
        self.acc_name = acc_name
        self.balance  = balance

    def __lt__(self, rhs):    # lt -> less than
        return self.balance < rhs.balance

x = [ BankAccount("120-2", "Somchai P.", 1000),
      BankAccount("110-1", "Boonchu K.",  500),
      BankAccount("320-9", "Saksri R.",  2000) ]
print( x[0] < x[1] )  # x[0].__lt__(x[1])
print( x[1] < x[2] )
s = sorted(x)
print(s)
```

sort, sorted, min, max อาศัย __lt__ ในการเปรียบเทียบ

```
False
True
[<__main__.BankAccount object at 0x026CA4B0>,
<__main__.BankAccount object at 0x02497CD0>,
<__main__.BankAccount object at 0x026CA4D0>]
```

แสดงแปลกๆ เพราะ print ไม่รู้จะแสดงอ็อบเจกต์อย่างไร

ผู้เขียนคลาสต้องออกแบบวิธีเปรียบเทียบอ็อบเจกต์

```
class BankAccount :
    ...
    def __lt__(self, rhs):
        return self.balance < rhs.balance
```

```
class BankAccount :
    ...
    def __lt__(self, rhs):
        return self.acc_no < rhs.acc_no
```

```
class BankAccount :
    ...
    def __lt__(self, rhs):
        return self.acc_name < rhs.acc_name
```

```
class BankAccount :
    ...
    def __lt__(self, rhs):
        return self.acc_name < rhs.acc_name or
               self.acc_name == rhs.acc_name and
               self.balance < rhs.balance
```

เมท็อด __str__ เพื่อแปลงอ็อบเจกต์เป็นสตริง

```
class BankAccount :
    def __init__ (self, acc_no, acc_name, balance):
        self.acc_no = acc_no
        self.acc_name = acc_name
        self.balance = balance

    def __lt__(self, rhs):
        return self.balance < rhs.balance

    def __str__(self):
        return "(Acct#:" + self.acc_no + \
               ", balance = "+ str(self.balance)+")"

x = [ BankAccount("120-2", "Somchai P.", 1000),
      BankAccount("110-1", "Boonchu K.",  500),
      BankAccount("320-9", "Saksri R.",  2000) ]
s = sorted(x)
print(s[0])
```

print อาศัย __str__ ในการแปลงอ็อบเจกต์เป็นสตริงเพื่อแสดง
print เรียกใช้ str
str เรียกใช้ __str__

```
(Acct#:110-1, balance = 500)
```

อ่านบัญชีธนาคาร เรียงตามยอดเงิน และแสดง

```
class BankAccount:
  def __init__ (self, acc_no, acc_name, balance):
    self.acc_no = acc_no
    self.acc_name = acc_name
    self.balance = balance
  def __lt__(self, rhs):
    return self.balance < rhs.balance
  def __str__(self):
    return "(Acct#:" + self.acc_no + \
                ", balance = "+ str(self.balance)+")"
def read_data(filename):
  infile = open(filename)
  accounts = []
  for line in infile:
    no,fname,lname,bal = line.split()
    accounts.append(BankAccount(no, fname+" "+lname, float(bal)))
  infile.close()
  return accounts

accounts = read_data("data_bank.txt")
accounts.sort()
for acc in accounts: print(acc)
```

```
010-123 John Smith      1000.00
010-213 Andrew Gibson   1500.00
010-312 Suthep Sawasdi   800.00
...
```

ตัวอย่าง : แสดงลำดับบัญชีธนาคารเรียงตามยอดเงิน

Input file

```
010-123 John Smith      1000.00
010-213 Andrew Gibson   1500.00
010-312 Suthep Sawasdi   800.00
020-292 Malai Malee      300.00
010-023 Goody Bayarami  1000.00
010-013 Nancy Tokai     1520.00
010-012 Somsri Sawasdi  8000.00
020-091 Somjai Noiboon  3000.00
010-112 Aree Yamahatai  1800.00
020-292 Joey Jelo        300.00
```

Output

```
(Acct#:020-292, balance = 300.0)
(Acct#:020-291, balance = 300.0)
(Acct#:010-312, balance = 800.0)
(Acct#:010-123, balance = 1000.0)
(Acct#:010-023, balance = 1000.0)
(Acct#:010-213, balance = 1500.0)
(Acct#:010-013, balance = 1520.0)
(Acct#:010-112, balance = 1800.0)
(Acct#:020-092, balance = 3000.0)
(Acct#:010-012, balance = 8000.0)
```

ลองทำดู

- ต่อยอดจากตัวอย่างข้างต้น
- เขียนโปรแกรมฝาก/ถอนเงินสดจากบัญชี

1) รอรับแป้นพิมพ์
   1) ถ้าจะฝากเงิน ให้ป้อน 1 หมายเลขบัญชี ยอดฝาก
   2) ถ้าจะถอนเงิน ให้ป้อน 2 หมายเลขบัญชี ยอดถอน
2) โปรแกรมจะแสดงผลยอดคงเหลือของบัญชีนั้น หลังจากเพิ่ม/ลดเงินในบัญชี
3) จากนั้นให้ย้อนมารอรับแป้นพิมพ์ใหม่ จนกว่าผู้ใช้ป้อน 0

ตัวอย่างเพิ่มเติม : Rational (จำนวนตรรกยะ)

```
class Rational:
  def __init__(self, n, d):
    self.numerator = n      # เศษ
    self.denominator = d    # ส่วน

  def __lt__(self, rhs):
    v1 = self.numerator/self.denominator
    v2 = rhs.numerator/rhs.denominator
    return v1 < v2

  def __str__(self):
    return str(self.numerator) + "/" +  \
           str(self.denominator)

r1 = Rational(2,4)
r2 = Rational(3,7)
print( r1 < r2 )
print( r1, r2 )
```

```
False
2/4 3/7
```

ตัวอย่างเพิ่มเติม : Date

```
class Date:
  def __init__(self, d, m, y):
    self.day = d
    self.month = m
    self.year = y

  def __lt__(self, rhs):
    if self.year != rhs.year:
      return self.year < rhs.year
    if self.month != rhs.month:
      return self.month < rhs.month
    return self.day < self.day

  def __str__(self):
    return str(self.day) + "/" +   \
           str(self.month) + "/" + \
           str(self.year)

d1 = Date(20,1,1990)
d2 = Date(9,12,1990)
print( d1 < d2 )
print( d1, d2 )
```

```
True
20/1/1990 9/12/1990
```

ตัวอย่างสุดท้าย : จงเขียน

- คลาส Date (เหมือนก่อนนี้)
- คลาส Person เก็บ วันเกิด เพศ ความสูง น้ำหนัก
  - เมท็อด
    - bmi : คืนค่า bmi ที่คำนวณจากน้ำหนักและความสูง
    - __lt__ : ให้บริการเปรียบเทียบน้อยกว่า โดยใช้ค่า bmi
    - __str__ : ให้บริการแปลงอ็อบเจกต์เป็นสตริง
- ฟังก์ชัน
  - read_data : อ่านข้อมูล Person จากแฟ้มมาเก็บ
  - get_genX : หาเฉพาะคนที่เป็น gen x
  - avg_bmi : หาค่า bmi เฉลี่ยของเพศที่กำหนดให้
  - sorted_by_birthday : เรียงลำดับตามวันเดือนเกิด
  - get_same_birthday : หาผู้ที่มีวันเดือนเกิดเดียวกัน

input / output

```
persons = read_data("persons.txt")
genx = get_genX(persons)
for p in genx:
  print(p)
```

```
(18/5/1976,M,175.0,75.0)
(6/4/1970,M,164.0,60.0)
(9/4/1969,M,172.0,73.0)
```

```
print(avg_bmi(persons,"M"))
```

```
24.40890039924406
```

```
for p in sorted(persons): #เรียงตาม bmi
  print(str(p) + " >> " + str(p.bmi()))
```

```
(7/8/1995,F,172.0,45.0)      >> 15.210924824229314
(1/1/1960,F,160.0,40.0)      >> 15.624999999999996
(30/11/1988,F,169.0,53.0)    >> 18.556773222226116
(24/6/1979,F,165.0,52.0)     >> 19.100091827364558
(6/4/1970,M,164.0,60.0)      >> 22.308149910767405
(7/8/1987,M,170.0,68.0)      >> 23.529411764705884
(18/5/1976,M,175.0,75.0)     >> 24.489795918367346
(9/4/1969,M,172.0,73.0)      >> 24.675500270416443
(30/11/1990,M,155.0,60.0)    >> 24.97398543184183
(30/11/1965,M,172.0,80.0)    >> 27.041644131963228
```

```
1/1/1960   F 160 40
30/11/1965 M 172 80
18/5/1976  M 175 75
24/6/1979  F 165 52
7/8/1987   M 170 68
7/8/1995   F 172 45
6/4/1970   M 164 60
9/4/1969   M 172 73
30/11/1990 M 155 60
30/11/1988 F 169 53
```

input / output

```
persons = read_data("persons.txt")
for p in sorted_by_birthday(persons):
  print(p)
```

```
(1/1/1960,F,160.0,40.0)
(6/4/1970,M,164.0,60.0)
(9/4/1969,M,172.0,73.0)
(18/5/1976,M,175.0,75.0)
(24/6/1979,F,165.0,52.0)
(7/8/1987,M,170.0,68.0)
(7/8/1995,F,172.0,45.0)
(30/11/1965,M,172.0,80.0)
(30/11/1990,M,155.0,60.0)
(30/11/1988,F,169.0,53.0)
```

```
1/1/1960   F 160 40
30/11/1965 M 172 80
18/5/1976  M 175 75
24/6/1979  F 165 52
7/8/1987   M 170 68
7/8/1995   F 172 45
6/4/1970   M 164 60
9/4/1969   M 172 73
30/11/1990 M 155 60
30/11/1988 F 169 53
```

```
for same_birthday in get_same_birthday(persons):
  out = [str(p) for p in same_birthday]
  print(",".join(out))
```

```
(7/8/1987,M,170.0,68.0),(7/8/1995,F,172.0,45.0)
(30/11/1965,M,172.0,80.0),(30/11/1990,F,155.0,60.0),(30/11/1988,F,169.0,5
```

ตัวอย่าง : Date

```
class Date:
  def __init__(self, d, m, y):
    self.day = d
    self.month = m
    self.year = y

  def __lt__(self, rhs):
    if self.year != rhs.year:
      return self.year < rhs.year
    if self.month != rhs.month:
      return self.month < rhs.month
    return self.day < self.day

  def __str__(self):
    return str(self.day) + "/" +   \
           str(self.month) + "/" + \
           str(self.year)

d1 = Date(20,1,1990)
print( d1 )
print( str(d1) )
```

```
20/1/1990
20/1/1990
```

Person

```
class Person :
    def __init__(self, birthdate, gender, height, weight):
        self.birthdate = birthdate  # Date object
        self.gender = gender        # "F" or "M"
        self.height = height        # in cm.
        self.weight = weight        # in kg.

    def bmi(self):
        return self.weight / (self.height/100)**2

    def __lt__(self, rhs):
        return self.bmi() < rhs.bmi()

    def __str__(self):
        return "(" + str(self.birthdate)+","+ \
                     str(self.gender)+","+ \
                     str(self.height)+","+ \
                     str(self.weight) + ")"
```

```
(6/4/1970,M,164.0,60.0)
```

read_data

```
def read_data(filename):
    infile = open(filename)
    # 10/1/1960 F 156 48
    persons = []
    for line in infile:
        date, gender, h, w = line.split()
        td,tm,ty = date.split("/")
        p = Person(Date(int(td),int(tm),int(ty)), \
                   gender, float(h), float(w))
        persons.append(p)
    return persons
```

```
1/1/1960   F 160 40
30/11/1965 M 172 80
18/5/1976  M 175 75
24/6/1979  F 165 52
7/8/1987   M 170 68
7/8/1995   F 172 45
6/4/1970   M 164 60
9/4/1969   M 172 73
30/11/1990 M 155 60
30/11/1988 F 169 53
```

get_genX, avg_bmi

```
def get_genX(persons):
    y1965 = Date(31,12,1965)
    y1977 = Date(1,1,1977)
    return [ p for p in persons \
                if y1965 < p.birthdate < y1977 ]


def avg_bmi(persons, gender):
    bmis = [ p.bmi() for p in persons
                     if p.gender == gender ]
    return sum(bmis)/len(bmis)
```

Generation X
Born: 1966-1976

Generation Y
Born: 1977-1994

Generation Z
Born: 1995-2012

http://www.socialmarketing.org/newsletter/features/generation3.htm

sorted_by_birthday

```
def sorted_by_birthday(persons):  # bubble sort
  s = list(persons)
  for k in range(len(s)-1):
    for i in range(len(s)-1):
      if  s[i].birthdate.month >  s[i+1].birthdate.month or \
          s[i].birthdate.month == s[i+1].birthdate.month and \
          s[i].birthdate.day > s[i+1].birthdate.day :
        s[i],s[i+1] = s[i+1],s[i]
  return s
```

ใช้เฉพาะวันกับเดือนในการเปรียบเทียบ

get_same_birthday

```
def get_same_birthday(persons):
    # input : [A, B, D, B, A, D, A, C]
    #  sorted [A, A, A, B, B, C, D, D]
    # output: [[A,A,A], [B,B], [D,D] ]

    s = sorted_by_birthday(persons)
    result = []
    k = 0
    for i in range(1,len(s)):
        if s[k].birthdate.day   != s[i].birthdate.day or \
           s[k].birthdate.month != s[i].birthdate.month:
            if i-k > 1: # ถ้ามีวันเดือนซ้ำ
                result.append(s[k:i])
            k = i
    if i-k >= 1: # ตรวจกรณีกลุ่มสุดท้าย
        result.append(s[k:])

    return result
```